



MakersSoc Introduction to Arduino

JJ Littleton

Arduino is open source design which uses a microcontroller. Micro controllers can be programmed to a range of tasks including taking inputs and writing outputs.

Equipment needed:

Arduino

USB B cable

Arduino IDE

(Personal) Toolkit

Breadboard

10k Ω Linear Potentiometer

LED x5

220 Ω resistors x5

Wire

Arduino

Arduino is an open-source electronics platform based on simple hardware and software. It is designed to make it easy for people to build and program interactive electronics projects.

The microcontroller in an Arduino is an Atmel AVR. A microcontroller is a small computer on a single integrated circuit that can be programmed to control various devices and perform different tasks.

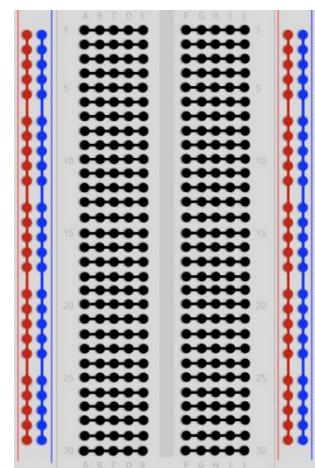
Arduino provides a simple, beginner-friendly environment for learning about electronics and programming, making it a great choice for anyone looking to get started in these fields.

Arduino is simply one of many microcontroller platforms.

How a breadboard is wired?

A breadboard is wired so that each numbered row is electronically connected alongside having power rails. To connect components, insert one leg into a row then another component's leg into the same row, the remaining legs should be in different rows as so they are not connected to themselves.

[1] Breadboard wiring





What are variables

Variables are a way to store and manipulate data in a program. In Arduino, variables can store different types of data such as numbers, characters and strings. Variables are declared with a type (e.g. `int`, `float`, `char`), a name and a value. For example: `int ledPin = 13;` declares a variable named "ledPin" of type "int" and assigns it the value "13".

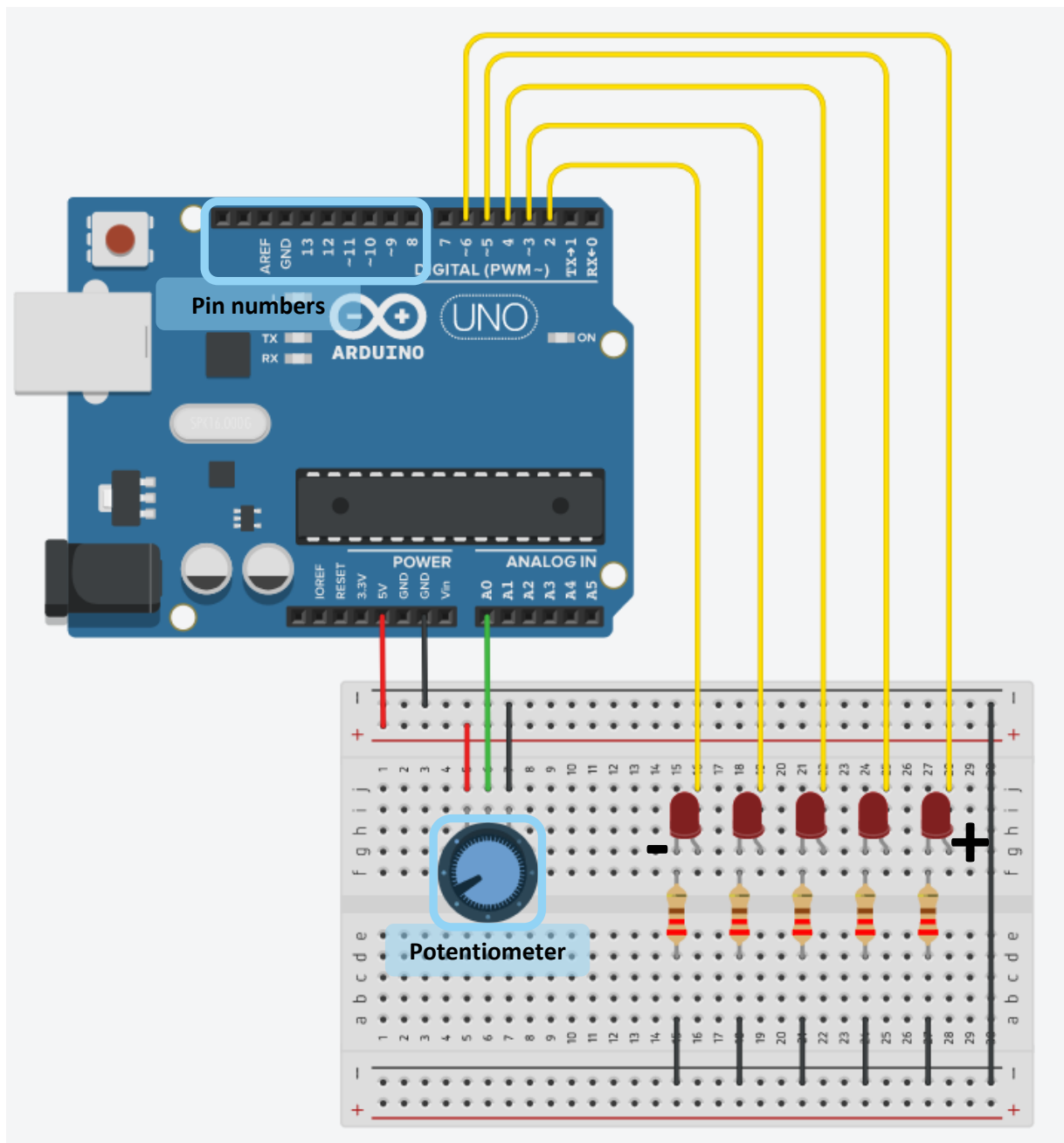
Variables are a fundamental concept in programming, and they are useful for several reasons:

1. **Data storage:** Variables allow you to store and manipulate data within a program. This is essential for most programs, as you need a place to keep information that you want to use or modify later.
2. **Dynamic programming:** By using variables, you can make your programs dynamic and changeable. For example, you can use variables to store user input, so that different users can run the same program with different results.
3. **Abstraction:** Variables allow you to abstract and encapsulate data, making your programs easier to understand and more modular. You can give variables meaningful names and use them to represent complex data structures, making your code more readable and maintainable.
4. **Reusability:** Variables make it possible to reuse code and data, as you can define variables once and use them in multiple parts of your program. This saves time and reduces the chance of introducing bugs into your code.

In short, variables are an important tool in programming because they allow you to store, manipulate and reuse data in a flexible and efficient way.



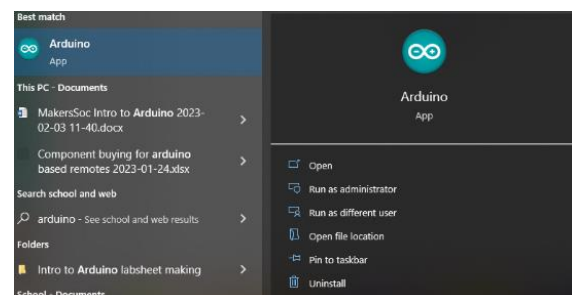
1. Assemble the circuit below



Do not forget the polarity of the LEDs. “Positively long legs”.

2. Start a new project

To start a new project, you need to open the Arduino Integrated Development Environment (IDE). If on your own computer, you can download the latest version of the IDE from the official Arduino website.





3. Create a new file

In the Arduino IDE, click on the "File" menu, then select "New." A new tab will open in the IDE with an empty sketch file.

The IDE may have already opened with a new file.

4. Write the following code:

```
void setup() {  
  // initialize serial communication at 9600 bits per second:  
  Serial.begin(9600);  
  int LED; // Define an integer variable "LED"  
}  
  
// the loop routine runs over and over again forever:  
void loop() {  
  // read the input on analog pin 0:  
  int sensorValue = analogRead(A0);  
  
  // Convert or 'map' the analog reading (which goes from 0-1023) to integers (2-6)  
  int LED = map(sensorValue, 0, 1023, 2, 6); //Mapping 0-1023 to 2-6  
  
  // print out the LED value you read:  
  Serial.println(LED);  
}
```

This has two functions: setup and loop.

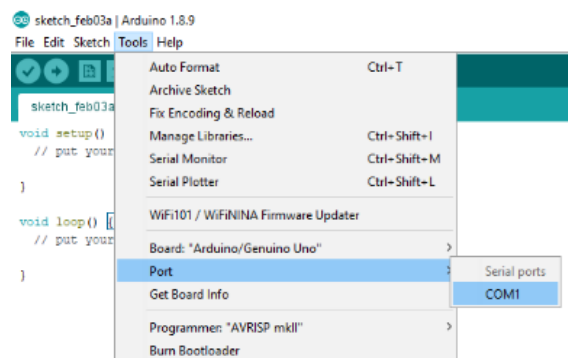
Setup is run upon start up.

Loop is then run and loops forever.

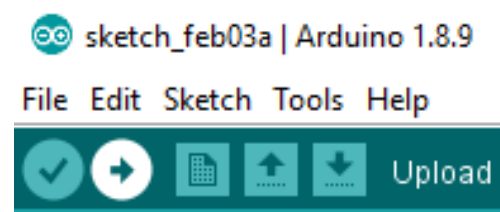
5. Upload to Arduino

Make sure the correct board and serial port are selected. To do this, go to the "Tools" menu, select "Board", and choose the appropriate board type. Then, go to the "Tools" menu, select "Serial Port", and choose the appropriate serial port.

There may be a list of options that need to be tried.



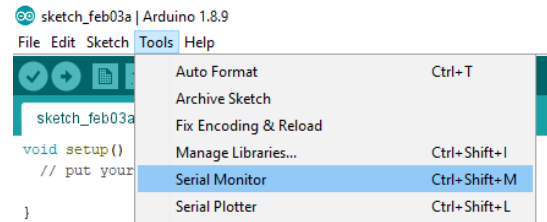
After selecting the correct port, attempt to upload to the board. This will load the program onto the Arduino.





6. Open the serial monitor

To do this, make sure the board is connected, go to the "Tools" menu and select "Serial Monitor".



7. Set output pins

Update your code with “pinMode(insert_pin_number_here, OUTPUT);” for each pin used, pins 2 to 6 inclusive.

Write this between Serial.begin and defining the variable LED within the setup function.

8. Writing to pins

Within loop(), write the following code at the end of the function.

```
//Using the range mapping as the pin number
digitalWrite(LED, HIGH); // Turn the relevant LED on
delay(100); // Wait 100 ms
digitalWrite(LED, LOW); // Turn the LED off
```

This instructs when the pin should be active or not. This is known as HIGH and LOW.



9. Your final code should look like this:

```
void setup() {  
  // initialize serial communication at 9600 bits per second:  
  Serial.begin(9600);  
  pinMode(2, OUTPUT); // Sets pin as output  
  pinMode(3, OUTPUT);  
  pinMode(4, OUTPUT);  
  pinMode(5, OUTPUT);  
  pinMode(6, OUTPUT);  
  int LED; // Define an integer variable "LED"  
}  
  
// the loop routine runs over and over again forever:  
void loop() {  
  // read the input on analog pin 0:  
  int sensorValue = analogRead(A0);  
  
  // Convert or 'map' the analog reading (which goes from 0-1023) to integers (2-6)  
  int LED = map(sensorValue, 0, 1023, 2, 6); //Mapping 0-1023 to 2-6  
  
  // print out the LED value you read:  
  Serial.println(LED);  
  
  //Using the range mapping as the pin number  
  digitalWrite(LED, HIGH); // Turn the relevant LED on  
  delay(100); // Wait 100 ms  
  digitalWrite(LED, LOW); // Turn the LED off  
}
```

10. Upload to Arduino

As you turn the potentiometer, the lit LED should move within the line of LEDs.

References

[1] R. Barnes, *Breadboard Wiring*. The MagPi, 2017.

Also see: "EE1000-Lab2-Microcontrollers.pdf" from Royal Holloway Electronic Engineering department. Used with permission.